

アイゼンハワーマトリクス TODO アプリ - 面接用プロジェクト説明書

📖 目次

- プロジェクト概要
- 技術スタック
- システムアーキテクチャ
- 主要機能の詳細
- API設計
- データ構造
- フロントエンド設計
- AI機能（Gemini統合）
- 工夫した点・技術的チャレンジ
- 想定される面接質問と回答例

1. プロジェクト概要

1.1 アプリケーション名

Eisenhower Board（アイゼンハワーボード）

1.2 コンセプト

アイゼンハワーマトリクス（Eisenhower Matrix）という時間管理手法をベースにした、社内向けタスク管理 Webアプリケーションです。

アイゼンハワーマトリクスとは？

元アメリカ大統領ドワイト・D・アイゼンハワーが考案した優先順位付けのフレームワークで、タスクを「重要度」と「緊急度」の2軸で4象限に分類します：

象限	分類	対応方針
第1象限	重要かつ緊急	🔥 今すぐ対応
第2象限	重要だが緊急ではない	🕒 計画的に進める
第3象限	緊急だが重要ではない	⚠️ 他者に任せる
第4象限	重要でも緊急でもない	🛑 手放しまたは休む

1.3 開発背景・目的

- 職員約10名程度のチームでタスクを可視化・共有したい
- 「誰が」「どのタスクを」「どれくらい急ぎ・重要か」を一目で把握
- 堅すぎないタスク管理ツールとして「遊び心」を取り入れる（表情変化など）

1.4 利用環境

- **対象ブラウザ:** Google Chrome 最新版
- **実行環境:** ローカル開発環境（将来的にクラウドデプロイ予定）
- **同時利用者:** 約10名程度

2. 技術スタック

2.1 バックエンド

技術	バージョン	用途
Python	3.11+	実行環境
FastAPI	0.115+	Webフレームワーク
Pydantic	v2	データバリデーション
Uvicorn	0.30	ASGIサーバー
Jinja2	3.1+	HTMLテンプレートエンジン
google-genai	1.53+	Gemini API SDK

2.2 フロントエンド

技術	用途
Vanilla JavaScript	UIロジック（ES Modules形式）
HTML5	マークアップ
CSS3	スタイリング（カスタムプロパティ活用）
Drag & Drop API	タスクカードの移動
Fetch API	バックエンドとの非同期通信

2.3 データ永続化

方式	ファイル	説明
JSONファイル	tasks.json	タスクデータ
JSONファイル	staff.json	職員データ

選定理由: v1では試験導入のため、外部データベースを使わずシンプルなファイルベースで実装。将来的にはPostgreSQL等への移行を想定。

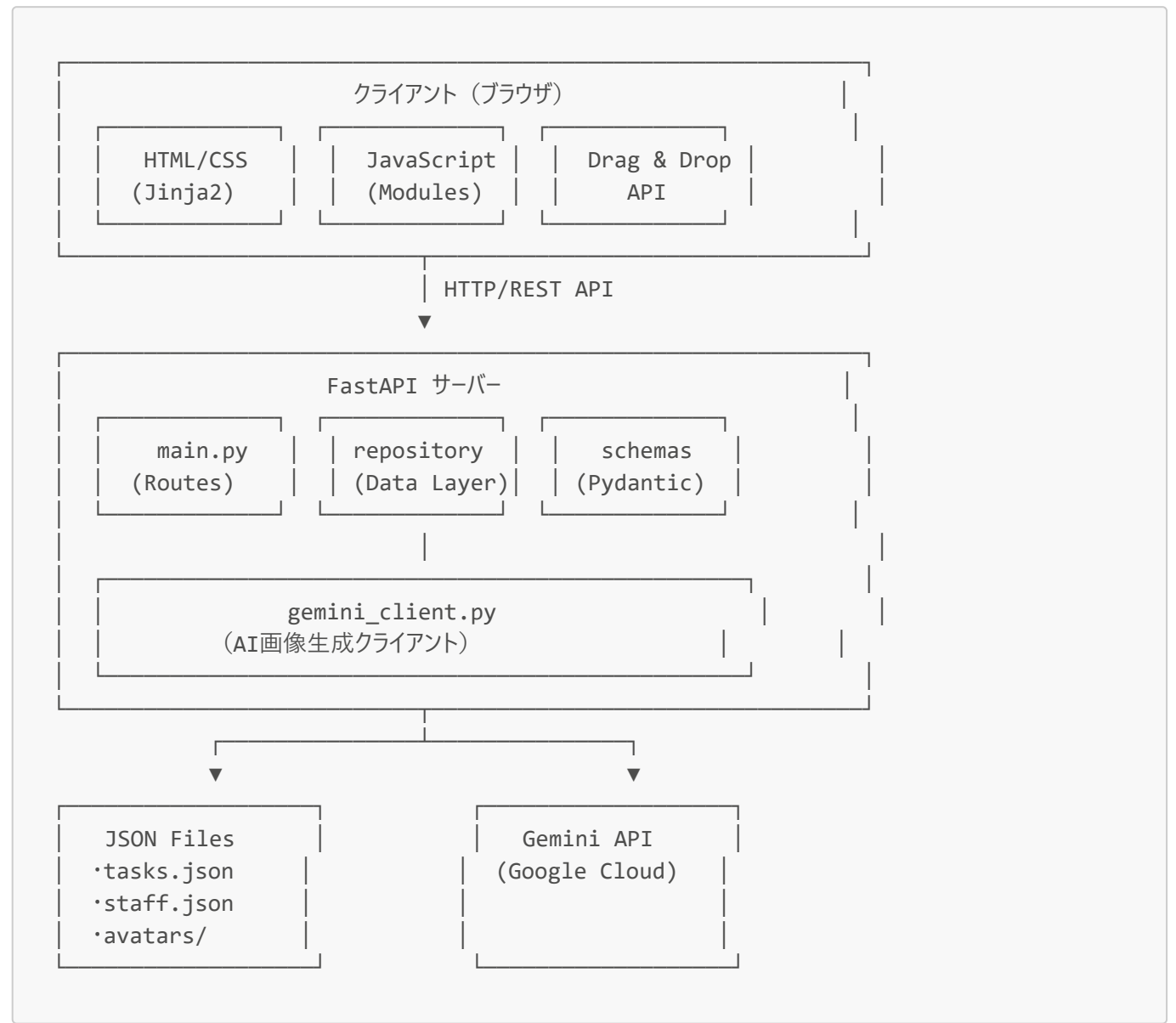
2.4 AI/ML

サービス	用途
------	----

サービス	用途
Google Gemini API	顔写真から4象限用表情アバターを自動生成

3. システムアーキテクチャ

3.1 全体構成図



3.2 ディレクトリ構成

```
アイゼンハワーTODO/
├── app/
│   ├── main.py           # FastAPIアプリケーション本体
│   ├── repository.py     # データアクセス層（JSONファイル操作）
│   ├── schemas.py        # Pydanticモデル定義
│   ├── gemini_client.py  # Gemini API クライアント
│   └── data/
│       ├── tasks.json    # タスクデータ
│       └── staff.json    # 職員データ
```

```
├── static/
│   ├── app.js           # メインJSファイル
│   ├── api.js           # API通信モジュール
│   ├── render.js        # レンダリングモジュール
│   ├── events.js        # イベントハンドラ
│   ├── state.js         # 状態管理
│   ├── utils.js         # ユーティリティ関数
│   ├── style.css        # スタイルシート
│   └── avatars/         # アバター画像保存先
├── templates/
│   └── index.html       # メインHTMLテンプレート
├── requirements.txt     # Python依存パッケージ
├── Procfile             # デプロイ設定
├── render.yaml          # Render.comデプロイ設定
└── 要件定義書.md       # 元の要件定義
```

4. 主要機能の詳細

4.1 アイゼンハワーマトリクスボード

機能概要:

- 4象限をグリッド表示し、各象限にタスクカードを配置
- 全職員が共有する1枚のボード

タスクカード表示項目:

- タイトル
- 担当者名とアバター画像
- ステータス（未着手/進行中/完了）
- 期限（締切日）
- 優先度（高/中/低）

4.2 ドラッグ&ドロップ機能

実装方法:

```
// HTML5 Drag & Drop API を使用
card.draggable = true;
card.addEventListener("dragstart", onDragStart);
quadrant.addEventListener("dragover", onDragOver);
quadrant.addEventListener("drop", onDrop);
```

処理フロー:

1. タスクカードをドラッグ開始
2. 別の象限にドロップ
3. バックエンドAPIを呼び出して象限を更新（PATCH /api/tasks/{id}/quadrant）

4. UIを即座に更新（楽観的UI更新）

4.3 タスクCRUD操作

操作	エンドポイント	メソッド
一覧取得	/api/tasks	GET
新規作成	/api/tasks	POST
更新	/api/tasks/{id}	PUT
象限移動	/api/tasks/{id}/quadrant	PATCH
削除	/api/tasks/{id}	DELETE

4.4 職員（スタッフ）管理

管理項目:

- 氏名
- 所属部署
- プロフィール写真（メイン + 4象限用表情画像）

4.5 複数ビュー表示

タブ名	説明
ボード	メインの4象限マトリクス表示
プレビュー	職員別タスク分布の概要表示
デンジャーリスト	第1象限（重要かつ緊急）のタスク一覧
暇人リスト	第4象限（重要でも緊急でもない）のタスク一覧
メンバー	職員一覧（検索・ソート機能付き）
完了	完了したタスクの一覧

5. API設計

5.1 RESTful API一覧

タスク関連

GET	/api/tasks	# タスク一覧取得
POST	/api/tasks	# タスク作成
PUT	/api/tasks/{task_id}	# タスク更新
PATCH	/api/tasks/{task_id}/quadrant	# 象限のみ更新
DELETE	/api/tasks/{task_id}	# タスク削除

スタッフ関連

GET	/api/staff	# スタッフ一覧取得
POST	/api/staff	# スタッフ作成（画像アップロード対応）
PUT	/api/staff/{staff_id}	# スタッフ更新
DELETE	/api/staff/{staff_id}	# スタッフ削除

Gemini AI関連（開発/テスト用）

POST	/api/gemini/test-image	# テキストから画像生成
POST	/api/gemini/edit-image	# 画像編集
POST	/api/gemini/four-expressions	# 4表情一括生成

5.2 リクエスト/レスポンス例

タスク作成

POST	/api/tasks	
Content-Type: application/json		
{		
	"title": "システム障害対応",	
	"description": "本番環境のエラー調査",	
	"owner_id": 1,	
	"created_by": "田中",	
	"due_date": "2025-12-30",	
	"status": "進行中",	
	"priority": "高",	
	"quadrant": 1	
}		

レスポンス（201 Created）：

{		
	"id": 10,	
	"title": "システム障害対応",	
	"description": "本番環境のエラー調査",	
	"owner_id": 1,	
	"owner": {	
	"id": 1,	
	"name": "田中一郎",	
	"department": "情報",	
	"photo": "3f9c4f6e-a28e-4024-b489-ac32d7805ad6.png",	
	"photo_q1": "203c4434-716a-436e-8c90-cad134638835_q1.png",	
	"photo_q2": "6575deca-556b-4059-9e40-dc3052507759_q2.png",	
}		

```
"photo_q3": "c04b34c4-8cfd-4b93-bf9a-49b06b3e582b_q3.png",
"photo_q4": "d8b1c2da-723a-4640-85eb-697cc8504b21_q4.png"
},
"created_by": "田中",
"due_date": "2025-12-30",
"status": "進行中",
"priority": "高",
"quadrant": 1
}
```

6. データ構造

6.1 タスク (Task)

```
class TaskBase(BaseModel):
    title: str = Field(..., max_length=80) # タイトル (必須)
    description: Optional[str] = Field(None, max_length=500) # 詳細説明
    owner_id: int # 担当者ID (必須)
    created_by: Optional[str] = None # 作成者名
    due_date: Optional[date] = None # 締切日
    status: Literal["未着手", "進行中", "完了"] = "未着手" # ステータス
    department: Optional[str] = None # 部署
    priority: Literal["高", "中", "低"] = "中" # 優先度
    quadrant: int = Field(1, ge=1, le=4) # 象限 (1-4)
```

6.2 スタッフ (Staff)

```
class Staff(BaseModel):
    id: int
    name: str
    department: Optional[str] = None
    photo: Optional[str] = None # 元のプロフィール写真
    photo_q1: Optional[str] = None # 第1象限用 (怒り)
    photo_q2: Optional[str] = None # 第2象限用 (やる気)
    photo_q3: Optional[str] = None # 第3象限用 (困惑)
    photo_q4: Optional[str] = None # 第4象限用 (リラックス)
```

6.3 JSONファイル構造例

tasks.json:

```
[
  {
    "id": 1,
    "title": "システム障害",
```

```
    "owner_id": 3,  
    "status": "進行中",  
    "priority": "高",  
    "quadrant": 1,  
    "due_date": "2025-12-30"  
  }  
]
```

staff.json:

```
[  
  {  
    "id": 1,  
    "name": "田中一郎",  
    "department": "情報",  
    "photo": "3f9c4f6e.png",  
    "photo_q1": "203c4434_q1.png",  
    "photo_q2": "6575deca_q2.png",  
    "photo_q3": "c04b34c4_q3.png",  
    "photo_q4": "d8b1c2da_q4.png"  
  }  
]
```

7. フロントエンド設計

7.1 JavaScriptモジュール構成

```
static/  
├─ app.js      # エントリーポイント、初期化处理  
├─ state.js    # アプリケーション状態管理  
├─ api.js      # バックエンドAPI通信  
├─ render.js   # DOM描画処理  
├─ events.js   # イベントハンドラ  
└─ utils.js    # 汎用ユーティリティ関数
```

7.2 状態管理

Vanilla JSでのシンプルな状態管理パターン:

```
// state.js  
let state = {  
  tasks: [],  
  staff: [],  
  editingId: null,  
  editingStaffId: null,  
  pendingDeleteId: null,  
}
```



```
completedSortOrder: "date-asc",
staffSortOrder: "department-asc",
staffFilterText: "",
};

// ゲッター/セッターでカプセル化
export function getEditingId() { return state.editingId; }
export function setEditingId(id) { state.editingId = id; }
```

7.3 レンダリング戦略

初期データのインライン埋め込み:

```
<script id="initial-data" type="application/json">
  {{ initial_payload | tojson }}
</script>
```

利点:

- 初回読み込み時のAPI呼び出し不要
- ページ表示が高速
- SEOフレンドリー（ただし本アプリでは重要度低）

7.4 ドラッグ&ドロップ実装

```
// カードのドラッグ開始
function onDragStart(event) {
  event.dataTransfer.setData("text/plain", taskId);
  event.dataTransfer.effectAllowed = "move";
}

// 象限へのドロップ
function onDrop(event) {
  event.preventDefault();
  const taskId = event.dataTransfer.getData("text/plain");
  const newQuadrant = parseInt(event.target.dataset.quadrant);

  // 楽観的UI更新（即座に画面を更新）
  updateTaskInUI(taskId, newQuadrant);

  // バックエンドに非同期で反映
  moveTaskAPI(taskId, newQuadrant);
}
```

8. AI機能（Gemini統合）

8.1 機能概要

メンバー追加時に1枚の顔写真をアップロードすると、Google Gemini APIを使って4象限ごとの表情画像を自動生成します。

8.2 4象限と表情の対応

象限	状況	生成される表情
Q1	重要かつ緊急	☹️ 怒っていて締め切りに追われている表情
Q2	重要だが緊急ではない	😏 やる気に満ちた計画的な表情
Q3	緊急だが重要ではない	😓 雑務に追われて困っている表情
Q4	重要でも緊急でもない	😌 お茶を飲んでリラックスした表情

8.3 実装の流れ

```
async def generate_quadrant_avatars(image_path, staff_name):
    # 1. 人物が含まれているかを検証
    is_person = await validate_person_image(image_path)
    if not is_person:
        raise GeminiAPIError("人物が検出されませんでした")

    # 2. 各象限用のプロンプトで画像を生成
    prompts = {
        "q1": "怒っていて締め切りに追われている表情にして",
        "q2": "やる気に満ちた計画的な表情にして",
        # ...
    }

    results = {}
    for key, prompt in prompts.items():
        image_data = await call_gemini_api(image_path, prompt)
        results[key] = image_data

    return results
```

8.4 環境変数設定

```
$env:GEMINI_API_KEY = "your-api-key"
$env:GEMINI_IMAGE_MODEL = "gemini-2.5-flash-image" # オプション
$env:GEMINI_ENABLED = "true" # 無効化するには "false"
```

8.5 エラーハンドリング

```
class GeminiConfigError(RuntimeError):
    """APIキー未設定などの設定エラー"""

class GeminiAPIError(RuntimeError):
    """API呼び出し時のエラー"""
```

- 設定エラーやAPI失敗時は、元の1枚の画像のみを使用
- ユーザー体験を損なわないよう、エラーを握りつぶして処理を継続

9. 工夫した点・技術的チャレンジ

9.1 シンプルなアーキテクチャ

課題: 小規模チームで素早く開発し、すぐに試験運用を開始したい

解決策:

- 外部データベースなしのJSONファイルベース永続化
- フレームワークに頼らないVanilla JSフロントエンド
- 依存関係を最小限に抑えた設計

メリット:

- 環境構築が容易（Pythonがあれば即起動可能）
- デバッグしやすい（JSONファイルを直接確認可能）
- 学習コストが低い

9.2 スレッドセーフなデータ操作

```
class DataRepository:
    def __init__(self):
        self._lock = Lock()

    def create_task(self, payload):
        with self._lock: # 排他制御
            data["id"] = self._next_task_id
            self._next_task_id += 1
            self._tasks.append(data)
            self._persist_tasks()
```

9.3 楽観的UI更新

ドラッグ&ドロップ時、API応答を待たずに即座にUIを更新することで、レスポンスな操作感を実現。

9.4 AI画像生成の段階的実装

1. まずテキストからの画像生成でAPI接続を確認
2. 画像編集機能を実装

3. 4表情一括生成を実装
4. スタッフ登録フローに統合

9.5 静的ファイルキャッシュ無効化ミドルウェア

開発中の利便性のため、静的ファイルのキャッシュを無効化:

```
class NoCacheMiddleware(BaseHTTPMiddleware):
    async def dispatch(self, request, call_next):
        response = await call_next(request)
        if request.url.path.startswith("/static/"):
            response.headers["Cache-Control"] = "no-cache, no-store"
        return response
```

10. 想定される面接質問と回答例

Q1: なぜアイゼンハワーマトリクスを選んだのですか？

回答例: 「タスクの優先順位付けは多くのチームで課題になっています。アイゼンハワーマトリクスは『重要度』と『緊急度』という2軸でシンプルに分類できる点が魅力でした。また、4象限それぞれに対応する表情を自動生成するというAI機能を組み合わせることで、視覚的にタスクの状況を把握しやすく、かつ遊び心のあるツールを実現できると考えました。」

Q2: FastAPIを選んだ理由は？

回答例: 「いくつかの理由があります。第一に、Pythonの型ヒントとPydanticを活用した自動バリデーションで、コードの品質と可読性を高められる点。第二に、非同期処理（async/await）がネイティブでサポートされており、Gemini APIの呼び出しなど非同期処理が多いこのアプリに適していました。第三に、自動生成されるOpenAPI仕様とSwagger UIにより、API仕様の文書化が不要になった点も開発効率の向上に貢献しました。」

Q3: なぜデータベースではなくJSONファイルを使ったのですか？

回答例: 「これはv1の試験運用という文脈での判断です。まず、約10名程度の小規模利用を想定しており、JSONファイルでも十分なパフォーマンスが得られること。次に、外部データベースへの依存を排除することで、環境構築のハードルを下げ、すぐに試用開始できること。また、JSONファイルを直接編集できるため、開発・デバッグが容易という利点もあります。将来的なスケールアップ時はPostgreSQLなどへの移行を想定しており、repository層を抽象化しているため、移行コストも最小限に抑えられます。」

Q4: セキュリティについてはどう考えていますか？

回答例: 「v1はローカル環境での社内利用を前提としているため、認証機能は実装していません。ただし、将来的な対応として以下を想定しています：

- ログイン機能の追加（FastAPIにはOAuth2サポートあり）

- APIキーなどの機密情報は環境変数で管理（コードにハードコードしない）
 - 入力値のバリデーションはPydanticで厳密に実施
 - CORS設定（必要に応じて特定オリジンのみ許可）」
-

Q5: ドラッグ&ドロップはどのように実装しましたか？

回答例: 「HTML5のDrag and Drop APIを使用しています。具体的には、タスクカードに `draggable="true"` 属性を付与し、`dragstart` イベントでタスクIDをDataTransferに設定します。各象限は `dragover` と `drop` イベントをリスンし、ドロップ時にPATCHリクエストでバックエンドを更新します。ユーザー体験を向上させるため『楽観的UI更新』を採用し、API応答を待たずに即座に画面を更新しています。」

Q6: Gemini AIをどのように活用していますか？

回答例: 「2つの主要な機能で活用しています。

1. **画像検証:** アップロードされた画像に人物が含まれているかをGeminiに判定させています。
2. **表情画像生成:** 元の顔写真から、4象限それぞれに対応した表情の画像を生成しています。プロンプトでは『同じ人物であることが分かるように顔立ちを保ちながら、表情だけを変更して』と指示しています。

技術的には`google-genai` SDKを使用し、`generate_content_stream`でストリーミングレスポンスを処理しています。エラー発生時はフォールバックとして元画像のみを使用し、ユーザー体験を損なわないようにしています。」

Q7: 今後の拡張計画は？

回答例: 「v2以降で以下の機能を検討しています：

- **認証・権限管理:** ログイン機能と、担当者のみ編集可能などの権限制御
 - **通知機能:** 期限が近いタスクのメール/Slack通知
 - **部署別ボード:** 部署ごとにボードを分ける機能
 - **データベース移行:** PostgreSQLへの移行とバックアップ体制の構築
 - **モバイル対応:** レスポンシブデザインの強化」
-

Q8: 開発で苦労した点は？

回答例: 「Gemini APIでの画像生成で苦労しました。特に以下の点です：

- **一貫性の確保:** 同じ人物として認識できるように表情だけを変更するのが難しく、プロンプトの調整を何度も行いました。
 - **エラーハンドリング:** API呼び出しは失敗する可能性があるため、4枚全ての生成が失敗した場合でもアプリが動作し続けるよう、エラー時のフォールバック処理を慎重に実装しました。
 - **非同期処理:** 4枚の画像を順番に生成するため時間がかかり、ユーザーに適切なフィードバックを提供する工夫が必要でした。」
-

Q9: テストはどのように行いましたか？

回答例:「手動テストを中心に行いました。特に:

- CRUD操作の正常系・異常系
- ドラッグ&ドロップの各象限間移動
- 画像アップロードとGemini生成
- JSONファイルの整合性確認

将来的には、pytestを使った自動テストの追加を検討しています。FastAPIはTestClientでAPIテストが容易に書けるため、主要なエンドポイントのテストを優先的に実装予定です。」

Q10: このプロジェクトで学んだことは？

回答例:「主に3つあります:

1. **シンプルさの価値:** 複雑な技術スタックより、チームが理解しやすいシンプルな構成の方が、開発速度と保守性の両面で有利なケースがあること。
 2. **AIとの統合:** Generative AIをアプリケーションに組み込む際の、プロンプトエンジニアリングやエラーハンドリングの重要性。
 3. **ユーザー体験の重要性:** ドラッグ&ドロップの楽観的UI更新のような、技術的には小さな工夫がユーザー体験を大きく向上させること。」
-

🔗 面接時のポイント

1. **デモを見せる:** 可能であれば実際に動作するアプリをデモする
 2. **技術選定の理由を明確に:** 「なぜその技術を選んだか」を論理的に説明
 3. **将来の拡張を意識:** 現状の制約を認識しつつ、将来の改善計画を示す
 4. **学びを伝える:** 技術だけでなく、プロジェクトを通じて得た学びを共有
-

Good luck with your interview! 🍀 面接頑張ってください！